PROJET D'IHM 2026 · CONCEPTION CENTRÉE UTILISATEUR

Modèles de tâches de la plateforme

Décomposition hiérarchique des tâches (HTA) de la plateforme d'apprentissage de la programmation par niveaux — Java · Python · C++

8 modèles | 4 acteurs | SEQ · ALT · ALT* · UNORD | boucles & optionnels | appliqué au projet

Ce que contient ce dossier

1. Objet et périmètre : pourquoi et comment on modélise les tâches de la plateforme

2. Les **huit tâches** retenues et le critère de sélection

3. La notation HTA employée, rappelée en une page

4. Les **huit modèles** : apprenant (A1–A4), enseignant (E1–E2), administrateur (AD1), agent IA (IA1)

5. Exploitation : boucles → exigences, matrice tâche → écran → exigence

6. Ce que les modèles changent dans la conception et ce qu'il reste à faire

Les huit modèles produits

A1 Créer son compte et choisir son parcours

A2 Étudier un cours et faire les exercices

A3 Passer le test de validation d'un niveau

A4 Suivre sa progression et reprendre après un échec

E1 Créer un niveau et fixer ses conditions de passage

E2 Suivre les résultats d'un niveau et l'ajuster

AD1 Administrer les comptes et les droits

IA1 Agent IA : assister l'apprenant sur un exercice

> **Règle de lecture :** un modèle de tâches décrit **ce que l'utilisateur doit faire**, jamais ce que le système fait à sa place. Chaque arbre se lit de haut en bas : le but, puis les procédures, puis les actions élémentaires.

1 · Objet et périmètre

La plateforme doit permettre à un apprenant de **progresser par niveaux dans trois langages** (Java, Python, C++), à un enseignant de **produire des niveaux et des règles de passage**, et à un administrateur de **gérer les comptes et les droits**. Avant de dessiner un seul écran, il faut savoir **ce que ces utilisateurs ont à faire** : c'est l'objet du modèle de tâches.

Ce dossier applique la démarche **HTA** (Hierarchical Task Analysis) à la plateforme elle-même. Il ne traite pas des exercices d'application du TP : ceux-ci ont seulement fourni la **notation** (rectangle, opérateurs, boucle, liaison pointillée) reprise ici.

1.1 Les entrées du travail

- L'**analyse des besoins v1.2** : 74 besoins fonctionnels, 35 non fonctionnels, 39 exigences IHM, 24 règles de gestion — chaque tâche modélisée cite les exigences qu'elle engage ;
- Les **acteurs** retenus : apprenant, enseignant, administrateur, plus l'**agent d'IA** (positionné en A et en B) qui n'est pas un acteur humain mais qui **agit** dans l'interface ;
- Les **trois parcours indépendants** (RG-20, RG-21) : un langage = un parcours = des niveaux, des seuils et une progression propres.

1.2 Ce qu'un modèle de tâches apporte ici

Question de conception	Réponse apportée par le modèle
Quels écrans faut-il ?	Un écran par tâche qui a des sous-tâches à regrouper : parcours, cours, éditeur de code, test, résultat, tableau de bord
Dans quel ordre ?	Le plan d'ordonnancement (SEQ / ALT / UNORD) donne la navigation et les fils d'Ariane
Où l'utilisateur se perd-il ?	Aux **boucles** (réessayer, corriger, repasser) et aux **alternatives** (quel langage, quelle action)
Que doit-on rendre visible ?	La condition d'arrêt de chaque boucle : niveau validé, tentatives restantes, code qui compile

Trois critères de sélection des tâches. On ne modélise pas tout : on retient les tâches **fréquentes** (elles structurent la navigation), **critiques** (une erreur coûte cher) et **complexes** (plusieurs branches, plusieurs acteurs).

1.3 La chaîne de valeur de l'apprenant



A1, A2 et A3 sont les tâches **fréquentes et structurantes** ; A4 est la tâche de **rattrapage**, celle qui décide de l'abandon ou de la persévérance. Côté enseignant, E1 précède E2 (on ne suit que ce qui a été créé). AD1 conditionne l'accès de tous, et IA1 traverse A2 et A3.

Hors périmètre assumé : les tâches purement techniques (sauvegarde, déploiement) et les tâches rares (réinitialiser un mot de passe) ne sont pas modélisées ici ; elles sont décrites dans l'analyse des besoins (BF-U6).

2 · Les huit tâches retenues

Code	Acteur	Tâche racine (but)	Fréquence	Criticité	Pourquoi elle est modélisée
A1	Apprenant	Créer son compte et choisir son parcours	une fois	forte	Premier contact : c'est là que se décide **le langage du parcours** (RG-20, RG-21) ; une erreur ici oriente toute la suite
A2	Apprenant	Étudier un cours et faire les exercices	quotidienne	forte	Le plus gros du temps passé : boucle d'exécution/correction, sauvegarde, aide de l'IA
A3	Apprenant	Passer le test de validation d'un niveau	par niveau	très forte	Tâche **centrale du sujet** : elle déclenche la validation et le déblocage du niveau suivant
A4	Apprenant	Suivre sa progression et reprendre après un échec	après chaque échec	forte	C'est la tâche où l'on renonce : elle doit rendre la sortie de boucle visible (tentatives restantes)
E1	Enseignant	Créer un niveau et fixer ses conditions de passage	par niveau	très forte	L'enseignant **produit la règle** que le moteur appliquera (RG-04, RG-05, BNF-09)
E2	Enseignant	Suivre les résultats d'un niveau et l'ajuster	périodique	moyenne	Boucle d'amélioration : un cours ne vaut que par les résultats qu'il produit (BF-S5)
AD1	Administrateur	Administrer les comptes et les droits	irrégulière	forte	Ouvre ou ferme l'accès aux autres tâches ; toute action est journalisée (BNF-06)
IA1	Agent IA	Assister l'apprenant sur un exercice	continue	moyenne à forte	Acteur non humain présent en positionnement **A** (à la demande) et **B** (accompagnement continu) : son périmètre doit être écrit noir sur blanc

2.1 Trois tâches ont un statut particulier

Tâche	Particularité retenue dans le modèle
A3	Seule tâche dont la sortie (**niveau validé**) n'est pas décidée par l'utilisateur mais par le **moteur de règles** : la boucle dépend donc d'un calcul serveur, pas d'une action d'écran
E1	Seule tâche où l'utilisateur **écrit une règle exécutable** : le **UNORD** des conditions (seuil, tentatives, période) impose un récapitulatif avant publication
IA1	Seule tâche portée par un **agent non humain** : elle se termine obligatoirement par une **restitution à l'humain**, car l'IA ne note pas, ne valide pas et ne débloque pas

> **Ce que ces huit modèles ne disent pas.** Ils décrivent les **buts et les procédures**, pas la nature des objets manipulés (cours, question, tentative). Cette couche est apportée par le **diagramme de classes** ; les deux se complètent : le modèle de tâches dit quand, le modèle de classes dit sur quoi.

3 · La notation employée

La notation est celle du cours de modèle de tâches, appliquée sans modification : c'est elle qui rend les huit arbres comparables entre eux.

Élément	Signification	Exemple pris dans ce dossier
Rectangle	Une tâche — un but à atteindre	« A3 · Passer le test de validation d'un niveau »
SEQ	Sous-tâches dans un ordre imposé	« Se préparer » puis « passer le test » puis « analyser le résultat »
ALT	Un seul choix possible	Parcours Java ou Python ou C++ ; un seul rôle par compte
ALT *	Alternative non stricte : plusieurs choix possibles	Cocher plusieurs cours à revoir, plusieurs indicateurs à afficher
UNORD	Aucun ordre imposé entre les sous-tâches	Seuil, tentatives et période de formation (E1) ; indicateurs de résultats (E2)
Boucle ↻	Répétition avec condition de répétition ou d'arrêt	« tant que le code ne compile pas » (A2, A3) ; « tant qu'il reste une tentative » (A4)
Pointillés	Sous-tâche optionnelle	« Demander une aide à l'IA », « Prévenir les apprenants », « Désactiver l'agent IA »



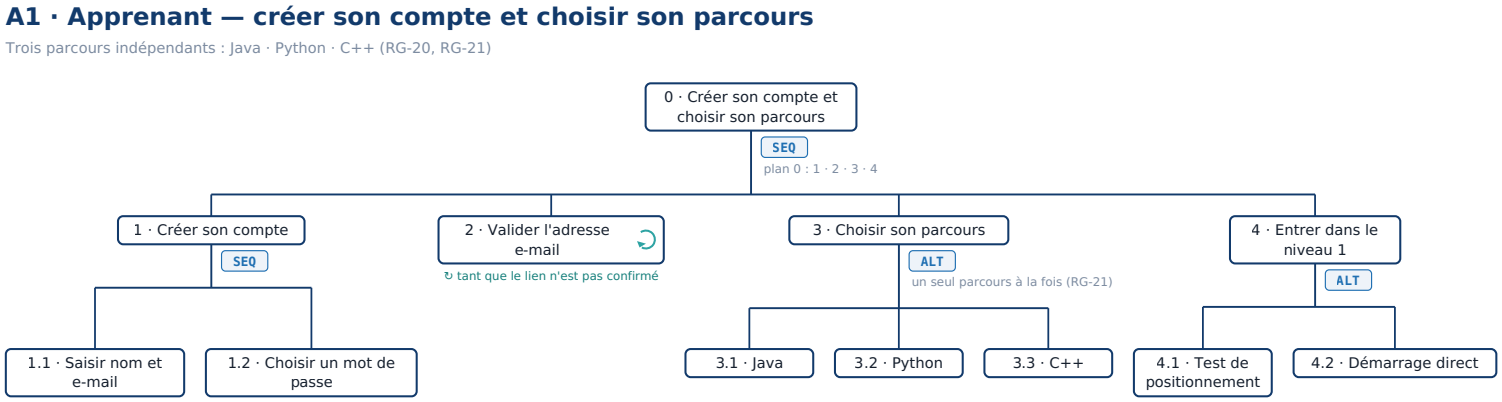
Convention de lecture adoptée dans tout le dossier : chaque tâche porte un **numéro de plan** (« plan 0 : 1 · 2 · 3 ») ; une tâche **sans décomposition** est une feuille, c'est-à-dire une action que l'utilisateur comprend sans explication supplémentaire (saisir, cliquer, lire). L'**opérateur s'écrit sur le lien**, la **condition de boucle s'écrit sous la boîte**.

4 · Les huit modèles de tâches de la plateforme

4.1 L'apprenant — quatre tâches, un même parcours

A1 ouvre le parcours, A2 le remplit, A3 le valide, A4 le rattrape. Les quatre arbres partagent le même postulat : **le langage est un choix de premier niveau** et il reste visible partout.

A1 Créer son compte et choisir son parcours — APPRENANT



Points remarquables : la validation de l'e-mail est une **boucle** (« tant que le lien n'est pas confirmé ») ; le choix du parcours est une **ALT** exclusive — un seul parcours à la fois, les autres restent accessibles plus tard (RG-21). L'entrée dans le niveau 1 est aussi une **ALT** : test de positionnement ou démarrage direct.
Exigences engagées : sélecteur de langage persistant (IHM-21), badge de langage (IHM-24), explication du verrouillage du niveau 2 (IHM-02), message d'erreur explicite à l'inscription (IHM-07).

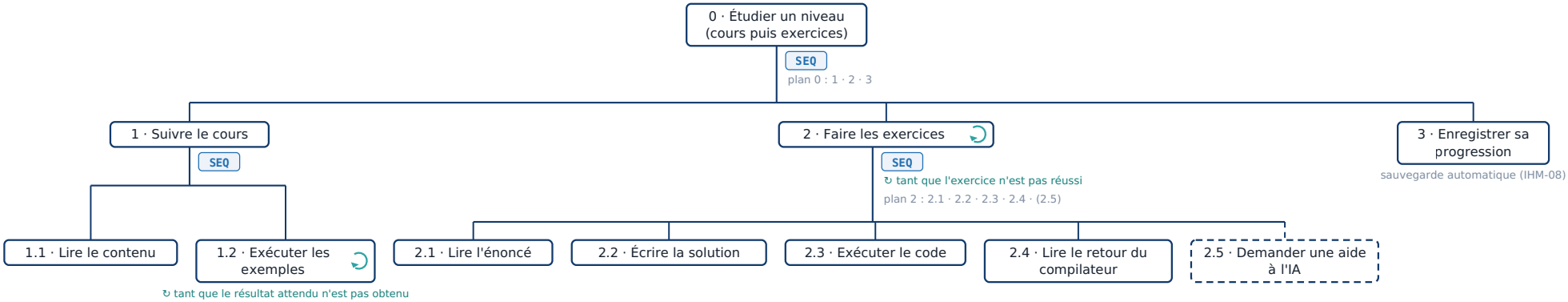
Tâche du modèle	Opérateur	Écran ou composant d'interface	Contrôle JavaFX (AtlantaFX + CSS3)
0 · Créer son compte et choisir son parcours	SEQ	Parcours d'inscription en 4 étapes avec barre de progression	`TabPane / barre d'étapes`
1 · Créer son compte	SEQ	Formulaire nom / e-mail / mot de passe + jauge de robustesse	`TextField + PasswordField + jauge`
2 · Valider l'adresse e-mail	boucle	Bandeau « e-mail non confirmé » + bouton « renvoyer le lien »	`bannière + Hyperlink`
3 · Choisir son parcours	ALT	Trois cartes Java / Python / C++ avec badge et version épinglée (RG-24)	`3 cartes HBox (.card-parcours)`
4 · Entrer dans le niveau 1	ALT	Écran « test de positionnement » ou bouton « démarrer le niveau 1 »	`2 boutons RadioButton`

A2 Étudier un cours et faire les exercices

APPRENANT

A2 · Apprenant — étudier un cours et faire les exercices

La boucle d'exercice occupe l'essentiel du temps : elle doit être la plus lisible



Points remarquables : deux boucles emboîtées — celle de l'exemple de code et celle de l'exercice (« tant que l'exercice n'est pas réussi »). Ce sont **les deux endroits où l'apprenant passe le plus de temps** ; l'aide de l'IA y est une sous-tâche **optionnelle** (pointillés). L'enregistrement de la progression est une tâche à part entière : elle doit être **automatique** et visible. **Exigences engagées :** éditeur avec exécution et réinitialisation (IHM-25), erreurs de compilation pédagogiques (IHM-26), sauvegarde automatique (IHM-08), réversibilité avant soumission (IHM-13), mode « sans assistant » activable par l'enseignant (BF-IA13).

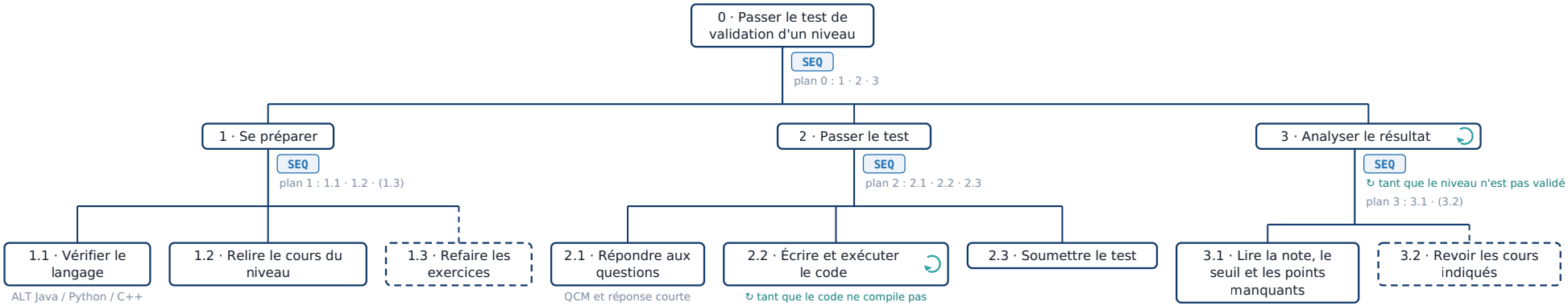
Tâche du modèle	Opérateur	Écran ou composant d'interface	Contrôle JavaFX (AtlantaFX + CSS3)
1 · Suivre le cours	SEQ	Lecteur de cours : blocs de code colorés + bouton « exécuter l'exemple »	`TreeView (sommaire) + zone de lecture`
2 · Faire les exercices	boucle	Éditeur de code + console (sortie réelle, erreurs ligne par ligne)	`TextArea coloré + console`
2.5 · Demander une aide à l'IA	option	Panneau latéral d'assistance, désactivable par niveau (BF-IA13)	`bouton de panneau latéral`
3 · Enregistrer sa progression	feuille	Indicateur discret « enregistré à 14:32 » (autosave, IHM-08)	`Label discret (autosave)`

A3 Passer le test de validation d'un niveau

APPRENANT

A3 · Apprenant — passer le test de validation d'un niveau

Tâche centrale : c'est elle qui débloque le niveau suivant (BF-R1, BF-R3)



Points remarquables : les questions forment un **UNORD** (l'ordre de réponse est libre) ; l'écriture du code est une **boucle** (« tant que le code ne compile pas ») ; l'analyse du résultat est elle-même **répétée** jusqu'à validation — c'est la boucle qui structure la vie de l'apprenant sur la plateforme. La révision des cours indiqués est **optionnelle**.
Exigences engagées : retour immédiat avec note, seuil et points manquants (IHM-03, BF-R1, BF-R3), nombre de tentatives et délai affichés, indication des cours à revoir (BF-R4, BF-IA3), reformulation du message par l'IA sans recalcul du chiffre (BF-IA8).

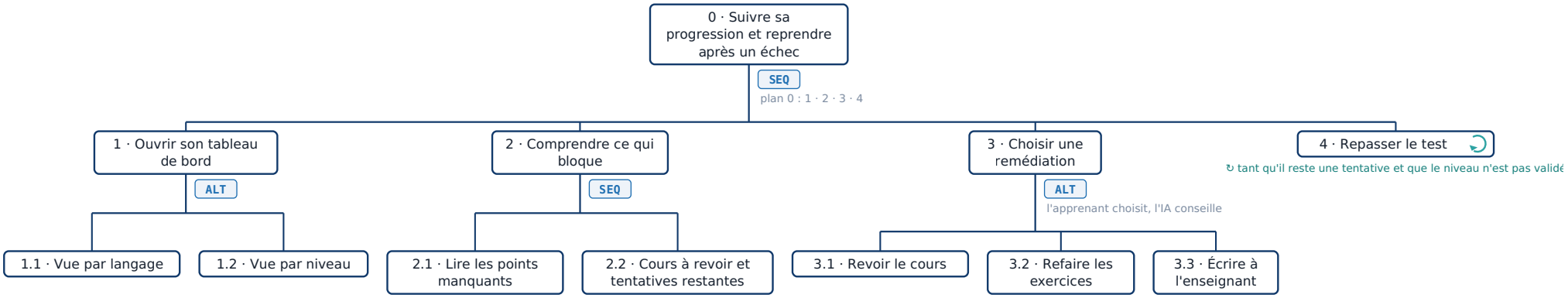
Tâche du modèle	Opérateur	Écran ou composant d'interface	Contrôle JavaFX (AtlantaFX + CSS3)
1 · Se préparer	SEQ	Frise du parcours + fiche du niveau : seuil, tentatives, période (IHM-02)	`ListView des niveaux + infobulle`
2 · Passer le test	SEQ	Écran de test : questions + éditeur, minuteur, bouton « soumettre »	`BorderPane : questions / éditeur / console`
2.1 · Répondre aux questions	UNORD	Navigation libre entre les questions, état « répondu » par question	`ListView + indicateur de réponse`
2.2 · Écrire et exécuter le code	boucle	Éditeur + console : erreur située (ligne, cause) — IHM-25, IHM-26	`TextArea + Console (streaming)`
3 · Analyser le résultat	boucle	Écran de résultat : note / seuil / points manquants / tentatives restantes	`grille de résultats + ProgressBar`

A4 Suivre sa progression et reprendre après un échec

APPRENANT

A4 · Apprenant — suivre sa progression et reprendre après un échec

La sortie de boucle doit toujours être visible : tentatives restantes (IHM-03)



Points remarquables : le tableau de bord propose deux vues en **ALT** (par langage / par niveau) ; la remédiation est une **ALT** à trois issues dont une **sortie vers l'humain** (écrire à l'enseignant) ; la boucle finale porte une double condition de sortie : **niveau validé** ou **plus de tentative**.
Exigences engagées : état du système toujours visible (IHM-01, BF-S1, BF-S2), parcours en frise avec états validé / en cours / verrouillé (IHM-05), raison chiffrée du verrou (IHM-02), trois tableaux de bord distincts (IHM-04).

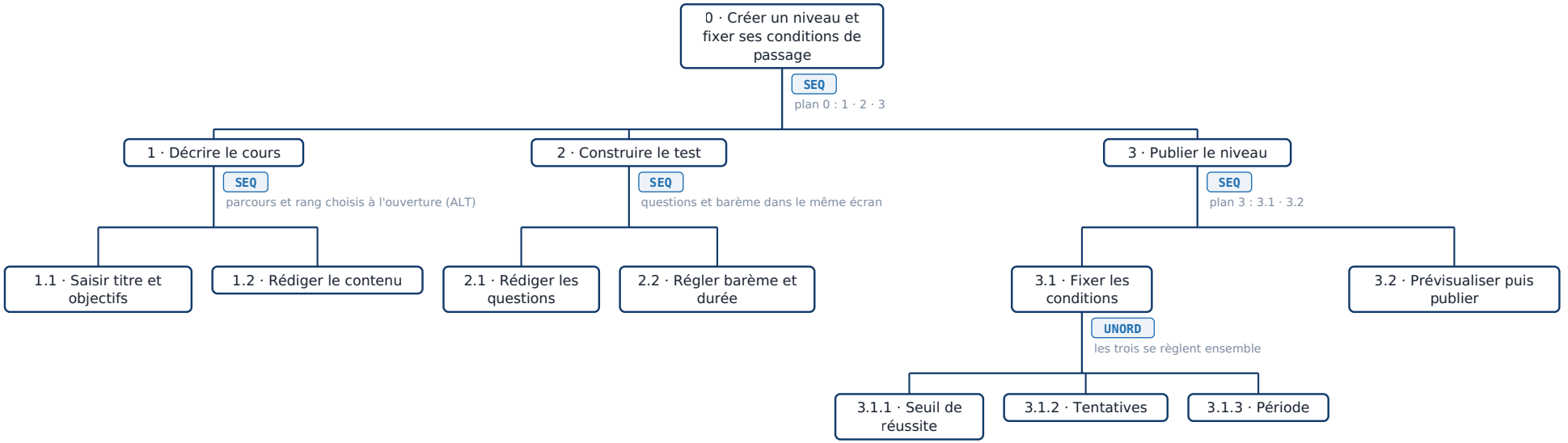
Tâche du modèle	Opérateur	Écran ou composant d'interface	Contrôle JavaFX (AtlantaFX + CSS3)
1 · Ouvrir son tableau de bord	ALT	Onglets « par langage » / « par niveau » (IHM-04, IHM-05)	`TabPane (2 vues) + frise Pane`
2 · Comprendre ce qui bloque	SEQ	Carte de résultats + liste des cours à revoir (BF-R4, BF-IA3)	`ListView + cartes de résultat`
3 · Choisir une remédiation	ALT	Trois actions : revoir / refaire / écrire à l'enseignant	`3 boutons d'action (.btn-remédiation)`
4 · Repasser le test	boucle	Bouton « nouvelle tentative » avec compteur et délai affichés	`bouton + Label compteur`

4.2 L'enseignant — créer la règle, puis la vérifier par les résultats

E1 Créer un niveau et fixer ses conditions de passage — ENSEIGNANT

E1 · Enseignant — créer un niveau et fixer les conditions de passage

C'est ici que s'écrit la règle que le moteur appliquera (RG-04, RG-05)



Points remarquables : le cours, puis le test, puis la publication ; les **conditions de passage sont en UNORD** (seuil, tentatives, période) — l'enseignant les règle dans l'ordre qu'il veut, ce qui impose un **récapitulatif avant publication**. Publier n'est pas un clic anodin : c'est le moment où la règle devient opposable aux apprenants.
Exigences engagées : éditeur confortable avec aperçu côte à côte (IHM-09, BF-C7, BF-C8), sauvegarde automatique du brouillon (IHM-08), aide contextuelle sur « seuil », « tentative », « période » (IHM-10), avertissement si un seuil modifié affecte des apprenants déjà validés (IHM-06), règles paramétrables sans redéploiement (BNF-09), prise en compte de **toutes** les conditions (BF-N7).

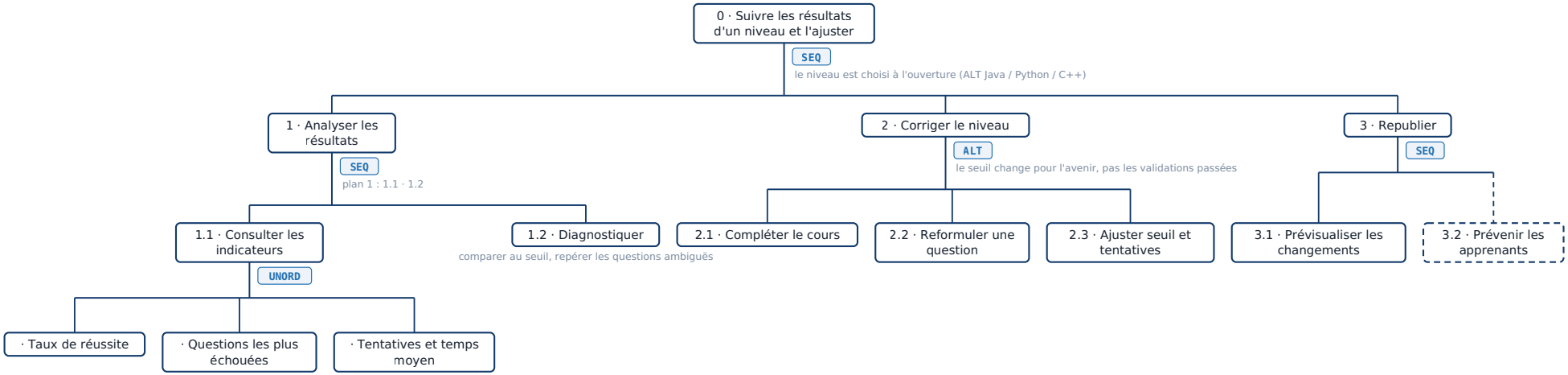
Tâche du modèle	Opérateur	Écran ou composant d'interface	Contrôle JavaFX (AtlantaFX + CSS3)
1 · Décrire le cours	SEQ	Éditeur de cours (titre, objectifs, blocs de code) + aperçu côte à côte	`SplitPane : éditeur / aperçu`
2 · Construire le test	SEQ	Éditeur de questions, barème et durée sur un même écran	`TableView des questions + Spinner`
3.1 · Fixer les conditions	UNORD	Trois champs : seuil, tentatives, période + récapitulatif	`Spinner + DatePicker`
3.2 · Prévisualiser puis publier	SEQ	Aperçu « vu par l'apprenant » + confirmation de publication	`aperçu + Alert de confirmation`

E2 Suivre les résultats d'un niveau et l'ajuster

ENSEIGNANT

E2 · Enseignant — suivre les résultats d'un niveau et l'ajuster

Boucle d'amélioration du cours ; aucune note n'est modifiée rétroactivement (RG-08)



Points remarquables : trois indicateurs en **UNORD** (taux de réussite, questions les plus échouées, tentatives et temps moyen) : l'enseignant commence par celui qui l'intéresse. La correction est une **ALT** (cours, question, conditions) et la prévention des apprenants est **optionnelle**. Le modèle pose une limite explicite : **un seuil modifié ne change pas les validations déjà acquises**.
Exigences engagées : tableau de bord d'avancement (BF-S5), historique des tentatives jamais écrasé (BF-E8), traçabilité des modifications de seuils et de notes (BNF-06), confirmation avant modification rétroactive (IHM-06).

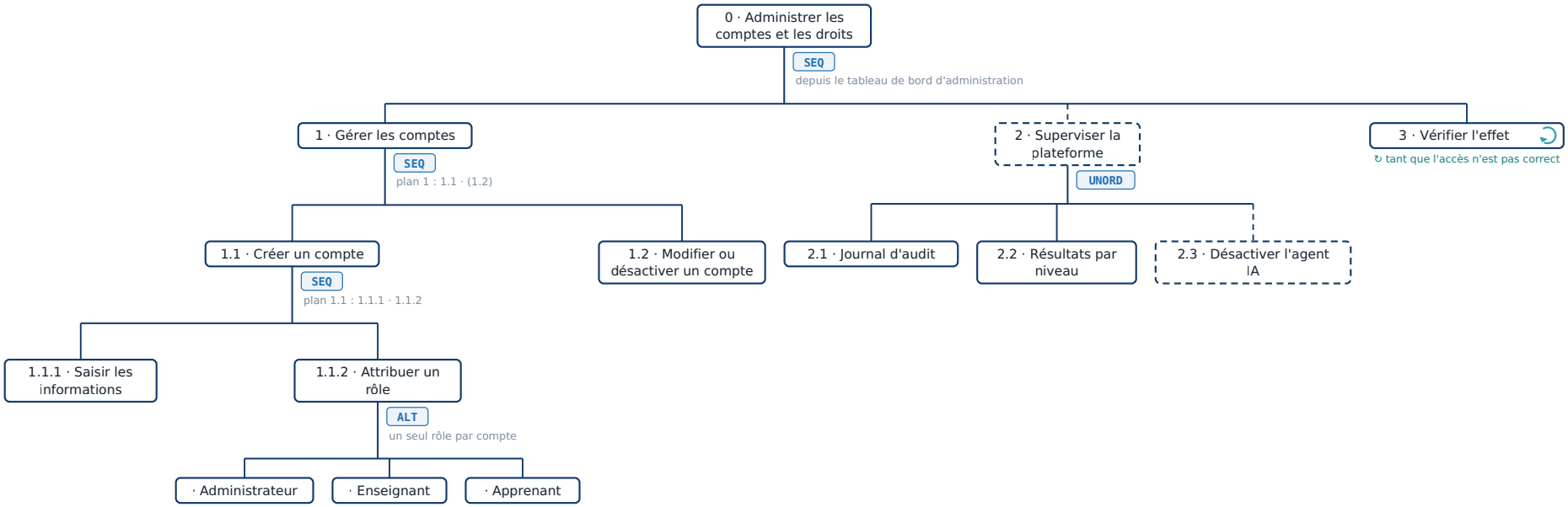
Tâche du modèle	Opérateur	Écran ou composant d'interface	Contrôle JavaFX (AtlantaFX + CSS3)
1.1 · Consulter les indicateurs	UNORD	Tableau de bord enseignant : 3 indicateurs + liste des apprenants	`TableView + 3 cartes KPI`
1.2 · Diagnostiquer	feuille	Comparaison seuil / résultats + questions les plus échouées	`Label comparatif + liste`
2 · Corriger le niveau	ALT	Accès direct au cours, à une question, ou aux conditions (IHM-06)	`TabPane (cours / question / conditions)`
3 · Republier	SEQ	Écran de diff « avant / après » + notification aux apprenants	`Alert de diff avant publication`

4.3 L'administrateur — ouvrir et fermer l'accès

AD1 Administrer les comptes et les droits — ADMINISTRATEUR

AD1 · Administrateur — administrer les comptes et les droits

Chaque action est journalisée et réversible (BNF-06, IHM-13)



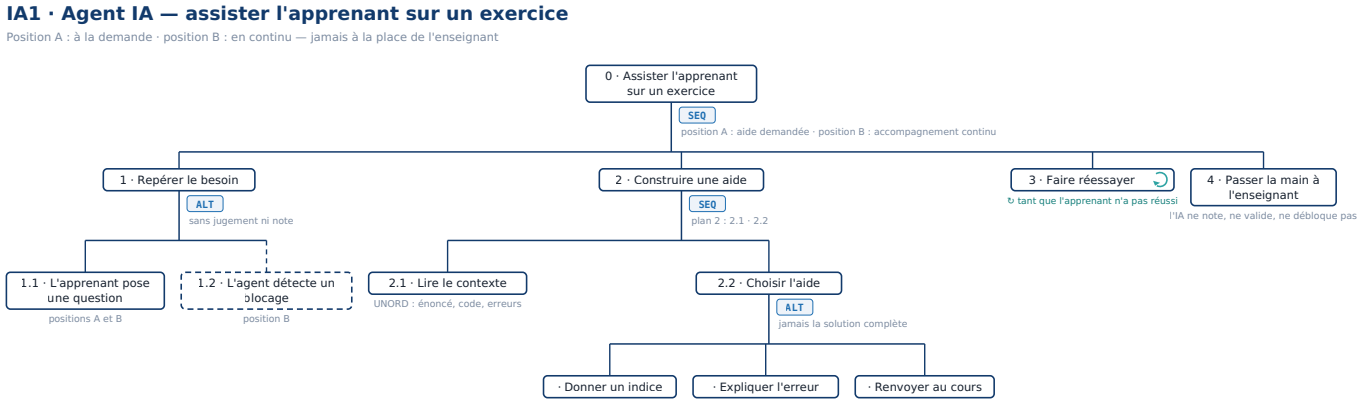
Points remarquables : l'attribution d'un rôle est une **ALT** imbriquée dans la création de compte (un seul rôle par compte) ; la supervision est une branche **optionnelle et non ordonnée** — l'admin y entre quand il veut, et peut y **désactiver l'agent IA** ; la vérification finale est une **boucle** (l'accès n'est pas correct → on corrige).
Exigences engagées : confirmation avant suppression (IHM-06), réversibilité — désactiver plutôt que supprimer (IHM-13), journal d'audit consultable (BNF-06), gestion du compte de service de l'agent IA avec rotation et révocation (BF-IA15, BNF-IA1).

Tâche du modèle	Opérateur	Écran ou composant d'interface	Contrôle JavaFX (AtlantaFX + CSS3)
1 · Gérer les comptes	SEQ	Table des utilisateurs : recherche, filtre par rôle, fiche de compte	`TableView + TextField de recherche`
1.1 · Créer un compte	SEQ	Formulaire d'identité + sélecteur de rôle unique	`Dialog de création + ComboBox de rôle`
2 · Superviser la plateforme	UNORD (opt.)	Journal d'audit, résultats par niveau, interrupteur « agent IA »	`TableView + boutons d'action`
3 · Vérifier l'effet	boucle	Bandeau de confirmation + test d'accès « se connecter en tant que »	`TitledPane (journal, résultats, IA)`

4.4 L'agent d'IA — un acteur non humain, un périmètre écrit

L'agent n'est pas un utilisateur : c'est un **acteur du système**. Le modéliser comme une tâche permet de fixer noir sur blanc **ce qu'il a le droit de faire**, dans les deux positionnements demandés : **A** — il répond quand on l'interroge, sans identité propre ni action hors session ; **B** — il agit en continu, avec un compte de service, un journal et des tâches planifiées.

IA1 Assister l'apprenant sur un exercice — AGENT IA



Points remarquables : le besoin est détecté soit parce que l'apprenant **pose une question** (positions A et B), soit parce que l'agent **repère un blocage** — sous-tâche **optionnelle**, disponible en position B seulement. L'aide est une **ALT** à trois issues qui exclut explicitement la solution complète. La phase « faire réessayer » est une **boucle** : l'agent ne fait pas à la place de l'apprenant. La dernière tâche — **passer la main à l'enseignant** — n'est pas optionnelle : c'est la garantie que **l'IA ne note pas, ne valide pas et ne débloque pas**.
Exigences engagées : panneau d'assistance distinct du contenu, désactivable par l'enseignant (BF-IA13), recommandations et reformulations pré-calculées (BF-IA3, BF-IA8), aucune écriture de l'agent dans les tentatives et les notes (BNF-IA15), désactivation possible par l'admin (BF-IA14).

Tâche du modèle	Opérateur	Écran ou composant d'interface	Contrôle JavaFX (AtlantaFX + CSS3)
1 · Repérer le besoin	ALT	Bouton « demander de l'aide » ; détection de blocage en position B	`bouton d'aide + détection serveur`
2.1 · Lire le contexte	feuille	Panneau latéral : contexte lu (énoncé, code, erreurs) affiché à l'apprenant	`panneau latéral (VBox défilante)`
2.2 · Choisir l'aide	ALT	Réponse courte : indice, explication de l'erreur, ou renvoi au cours	`3 boutons de réponse courte`
3 · Faire réessayer	boucle	Rappel « à toi d'essayer » + retour au code, jamais de solution complète	`rappel + retour au TextArea`
4 · Passer la main à l'enseignant	feuille	Bouton « transmettre à l'enseignant » + copie de l'échange	`bouton + Alert de transmission`

5 · Ce que les modèles imposent à l'interface

5.1 Les boucles sont les points de rupture : six boucles, six exigences

Chaque boucle du modèle devient un **point de rupture** de l'interface : c'est là que l'utilisateur ignore s'il doit continuer ou s'arrêter. Une boucle dont la condition de sortie n'est pas affichée est un abandon programmé.

Boucle du modèle	Condition de sortie	Risque si l'interface ne la montre pas	Réponse d'interface
A1 · validation de l'e-mail	lien confirmé	Compte créé mais inaccessible, l'utilisateur recommence	Bandeau « e-mail non confirmé » + renvoi du lien (IHM-07)
A2 · exécution d'un exemple	résultat attendu obtenu	L'apprenant croit son environnement cassé	Sortie réelle + différence attendu/obtenu (BF-L6)
A2 · exercice réussi	tous les cas passent	Essais en aveugle, abandon	Messages de compilation pédagogiques (IHM-26), tests visibles
A3 · code qui compile	compilation sans erreur	Blocage avant même de pouvoir soumettre	Erreur située (ligne, cause) et bouton « exécuter » (IHM-25)
A3 · niveau validé	seuil et conditions satisfaits	Impression d'impasse après un échec	Note, seuil, points manquants, tentatives restantes (IHM-03, BF-R3)
A4 · nouvelle tentative	validé **ou** plus de tentative	L'apprenant ne sait pas s'il peut recommencer	Compteur de tentatives et délai affichés en permanence (IHM-01)

5.2 Répartition des opérateurs : ce que les huit arbres disent de la plateforme

Comptée sur les huit arbres, cette répartition indique **où la complexité se concentre**. Deux constats en sortent : ① **six des huit boucles** appartiennent aux modèles de l'apprenant — c'est là que se joue la persévérance, donc là que l'interface doit le plus travailler ; ② les modèles de l'enseignant et de l'administrateur portent surtout des **alternatives et des non-ordonnés** : leur difficulté n'est pas la répétition mais le **choix** — d'où les récapitulatifs, aperçus et confirmations exigés en E1, E2 et AD1.

Modèle	Boîtes	SEQ	ALT	ALT *	UNORD	Boucles ↻	Optionnels	Où se situe la difficulté
A1	12	2	2	0	0	1	0	décisions : langage, entrée
A2	11	3	0	0	0	2	1	répétition : exercice
A3	12	4	0	0	0	2	2	répétition : validation
A4	12	2	2	0	0	1	0	répétition : rattrapage
E1	13	4	0	0	1	0	0	paramétrage sans ordre
E2	14	3	1	0	1	0	1	lecture libre d'indicateurs
AD1	14	3	1	0	1	1	2	choix d'action + optionnels
IA1	12	2	2	0	0	1	1	choix de l'aide, boucle courte

5.3 Matrice tâche → écran → exigence

Cette matrice est le livrable de sortie du modèle de tâches : elle transforme chaque arbre en **une liste d'écrans à maquetter** (IHM-20) et indique, pour chacun, les exigences qu'il doit satisfaire. Une ligne sans écran serait une tâche oubliée ; un écran sans ligne serait un écran inutile.

Code	Tâche (niveau 1 et 2)	Écran nécessaire	Exigences couvertes
A1	Créer son compte · choisir son parcours	Formulaire d'inscription + écran « choisir mon parcours » (3 cartes Java / Python / C++)	IHM-21, IHM-24, IHM-07, RG-20, RG-21
A2	Suivre le cours · faire les exercices	Lecteur de cours à blocs de code + éditeur de code avec sortie	IHM-25, IHM-26, IHM-08, IHM-13, BF-L5, BF-L6, BF-IA13
A3	Se préparer · passer le test	Frise du parcours + écran de test (questions + éditeur) + écran de soumission	IHM-02, IHM-05, IHM-11, BF-N7, BF-E8
A3	Analyser le résultat	Écran de résultat détaillé (note, seuil, points manquants, cours à revoir)	IHM-01, IHM-03, BF-R1, BF-R3, BF-R4, BF-IA3, BF-IA8
A4	Tableau de bord · remédiation	Tableau de bord apprenant (deux vues : par langage / par niveau)	IHM-01, IHM-04, IHM-05, BF-S1, BF-S2
E1	Décrire le cours · construire le test · publier	Éditeur de cours, éditeur de questions, formulaire des conditions, aperçu	IHM-08, IHM-09, IHM-10, IHM-06, BF-C7, BF-C8, BNF-09
E2	Analyser les résultats · corriger	Tableau de bord enseignant (indicateurs + liste des apprenants)	BF-S5, BF-E8, BNF-06, IHM-06
AD1	Gérer les comptes · superviser	Tableau des utilisateurs et des rôles + journal d'audit	BF-U3, BF-U5, BNF-06, IHM-06, IHM-13, BF-IA15
IA1	Assister l'apprenant	Panneau d'assistance latéral (indices, explication d'erreur, renvoi au cours)	BF-IA13, BF-IA3, BF-IA8, BNF-IA15, BF-IA14

Ce que la matrice démontre : les huit modèles produisent **neuf familles d'écrans** pour quatre acteurs, et chaque écran est rattaché à des exigences déjà écrites dans l'analyse des besoins v1.2 — aucune exigence « inventée » pour les besoins du modèle de tâches. Les écrans les plus chargés en exigences sont l'**écran de résultat** (A3) et l'**éditeur d'exercice** (A2) : ce sont donc eux qu'il faut maquetter et tester en premier (IHM-20).

5.4 De l'arbre au contrôle JavaFX : la traduction technique

Le projet est développé en **JavaFX**, habillé par la bibliothèque de thèmes **AtlantaFX** et stylé en **CSS3** (analyse des besoins v1.3, §1.1 et §6.3). Chaque tâche de ces arbres a donc **deux adresses** : un écran dans la matrice 5.3, et un **contrôle JavaFX** dans le code.

Modèle	Écran principal	Contrôles JavaFX dominants	Feuille CSS3 / thème
A1	Inscription + choix du parcours	TabPane · TextField · 3 cartes HBox	theme-light / theme-dark · .card-parcours
A2	Lecteur de cours + exercice	TreeView · TextArea · Console · ProgressIndicator	.editor · syntax-java / -python / -cpp
A3	Test de validation	BorderPane · ListView · Spinner · ProgressBar	.test-mode (assistant masqué, IHM-IA8)
A4	Tableau de bord apprenant	TabPane · Pane (frise) · InfoTooltip	.frise · .etat-valide / -en-cours / -verrouille
E1	Éditeur de niveau	SplitPane · TableView · Spinner · DatePicker	.editor · .conditions (récapitulatif)
E2	Tableau de bord enseignant	TableView · cartes KPI · Alert de diff	.kpi · .etat-alerte
AD1	Administration	TableView · Dialog · ComboBox · TitledPane	.admin · .action-danger
IA1	Panneau d'assistance	VBox défilante · boutons courts · Label sourcé	.panel-ia (masquable par classe)

5.5 Les jetons de design, écrits une seule fois

Les couleurs des trois langages et les états des niveaux sont déclarés **dans une seule feuille** (IHM-JFX3, BNF-T4) : le Java ne contient aucune couleur.

```
/* app.css — jetons de design */
.root {
  -color-java:   #b06a00;   /* badge + accents */
  -color-python: #1f6fb2;
  -color-cpp:    #17807c;
  -radius-card:  8px;
  -space-2:      8px;
}
.badge-langage { -fx-background-radius: 12px; }
.card-niveau.etat-valide     { -fx-border-color: -color-cpp; }
.card-niveau.etat-verrouille { -fx-opacity: .55; }
```

Le thème clair / sombre est fourni par AtlantaFX : basculer de thème = changer **une classe sur la scène**, sans recompiler (IHM-JFX2).

5.6 Trois disciplines non négociables

Discipline	Conséquence pour le développement JavaFX
Style externalisé	0 appel à `setStyle()` non justifié : tout passe par une classe CSS (BNF-T4). Une revue de code le vérifie.
Décision côté serveur	Un niveau verrouillé reste verrouillé même si l'on modifie le client : c'est l'API qui refuse (BNF-T6, BNF-01).
Code exécuté à distance	JavaFX **n'exécute jamais** le code de l'apprenant : il envoie le source, affiche la sortie en streaming (BNF-03, BNF-L1).

> **Ce que cela change dans les arbres.** Les boucles « tant que le code ne compile pas » (A2, A3) et « tant que l'apprenant n'a pas réussi » (IA1) ne sont pas des boucles de l'interface : ce sont des **allers-retours avec le serveur**. L'IHM doit donc afficher un **état d'attente** à chaque tour (BNF-T3) et non un simple blocage.

6 · Bilan et suite du travail

6.1 Ce que les modèles changent dans la conception

- La plateforme n'est pas « un site avec des cours » : c'est un **enchaînement de décisions** — choisir un langage, réussir un exercice, franchir un seuil, reprendre après un échec ;
- **Trois boucles gouvernent l'expérience** : la boucle d'exercice (A2), la boucle de validation (A3) et la boucle de rattrapage (A4). Toute l'ergonomie consiste à rendre leur condition d'arrêt visible ;
- Le **langage** est un objet de premier niveau : il apparaît dès A1 et doit rester affiché en permanence (IHM-21, IHM-24) ;
- L'**agent IA** est modélisé comme une tâche bornée : il aide, il ne décide pas. Sa dernière sous-tâche est un passage à l'humain, ce qui traduit la règle « l'IA ne note, ne valide, ne débloque pas » ;
- Les **tâches de l'enseignant** produisent les règles que subissent les apprenants : d'où l'exigence de récapitulatif, d'avertissement et de traçabilité autour de la publication (E1).

6.2 Ce qui reste à faire

Quatre prolongements, non traités dans ce dossier :
① dériver de ces arbres les **maquettes FXML** des écrans de la matrice 5.3, à dessiner dans Scene Builder et à styler avec AtlantaFX + CSS3 (IHM-20, IHM-JFX1) ;
② décrire les mêmes tâches en **CTT / KTAM** pour traiter la **concurrence** (par exemple l'agent IA qui intervient pendant l'exercice) — HTA ne modélise pas le parallélisme ;
③ écrire les **scénarios de test utilisateur** en suivant A3 et A4 : ce sont les deux tâches où les boucles se referment ;
④ vérifier la **couverture** : chaque tâche feuille doit avoir une action, un écran ou un message.

Conclusion. Huit arbres suffisent à couvrir la plateforme du premier clic à la validation d'un niveau, pour les trois rôles humains et pour l'agent d'IA. Ils transforment l'analyse des besoins en **parcours vérifiables** : chaque boîte de ces arbres peut être confrontée à une exigence (IHM, BF, BNF, RG) et à un écran. C'est la charnière entre le cahier des charges et les maquettes.